TP8 - Infraestructura como código de PhotoMatch

1. Objetivo y teoría

Terraform declara el estado deseado. **plan** compara código, estado y recursos; **apply** converge; **destroy** elimina sólo lo administrado. El estado contiene identificadores y también secretos, por lo que no se versiona. Idempotencia significa que un segundo plan no cambia nada.

2. Situación inicial

QA/PROD del fallback se diseñaron como proyectos Compose. No había carpeta **infra/** ni preprod. La consigna exige un tercer entorno aislado que nazca y muera frente al docente.

3. Archivos modificados

infra/versions.tf, **variables.tf**, **main.tf**, **outputs.tf**, **.terraform.lock.hcl** y **.gitignore**.

4. Código importante

main.tf declara red, volumen, tres imágenes y tres contenedores: ocho recursos. La URL PostgreSQL se deriva de **docker_container.db.name**; **BACKEND_URL** se deriva de **docker_container.backend.name**. Las imágenes backend/frontend entran como variables para usar los tags **sha-<commit>** publicados. Las claves llegan por **TF_VAR_*** sensibles. El puerto 3003 no toca QA 3001 ni PROD 3002.

5. Decisiones y alternativas

Se usó el provider Docker 4.6.0 y Terraform 1.17.x, ambos fijados, para evitar cuentas de Neon/Render. El estado vive fuera del repo. No se importan ni se destruyen QA/PROD. Las imágenes locales se usaron sólo para probar el HCL antes del merge; para cerrar TP8 se repetirá con GHCR real.

6. Herramientas

terraform init descarga provider y fija checksums; **fmt** ordena HCL; **validate** revisa esquema; **plan** muestra cambios; **apply** crea; **destroy** elimina. Docker Desktop provee daemon y el provider usa **DOCKER_HOST** del contexto actual.

7. Tests y evidencia real

init, **fmt -check** y **validate** pasaron. Primer plan: 8 altas. Apply creó preprod; **/health** respondió con PostgreSQL. Segundo plan: sin cambios. Se ejecutó **destroy**, un segundo **apply**, otro **/health** y **destroy** final. Preprod quedó ausente. **Falta** repetir con imágenes GHCR de TP7, revisar QA/PROD y publicar **v8.0.0**.

8. Problemas y solución

En macOS el provider buscó **/var/run/docker.sock** y falló. **export DOCKER_HOST="$(docker context inspect --format '{{.Endpoints.docker.Host}}')"** apuntó al socket de Docker Desktop. El contenedor web devolvió 502 durante el arranque; el retry posterior pasó cuando API y DB estuvieron listas.

9. Preguntas del profesor

¿Dónde vive el estado? En **~/.local/share/photomatch-tfstate/preprod.tfstate**, fuera del repo. **¿Qué significa known after apply?** El provider aún no conoce el ID hasta crear. **¿Qué es drift?** Diferencia entre estado declarado y real. **¿Por qué no tocar QA/PROD?** Preprod debe convivir y su destroy no puede afectarlos.

10. Demo de defensa

Con preprod destruido, exportar **DOCKER_HOST** y variables secretas localmente, correr **terraform plan** y explicar ocho altas, **apply**, abrir **http://127.0.0.1:3003/health** y login, segundo plan sin cambios y **destroy**. No mostrar valores secretos ni contenido del estado en pantalla.